

Curia-Lite: Minimizing Model Size for Keyword and Temporal Query Extraction

Will Adair

University of Nebraska – Lincoln
Email: wadair2@huskers.unl.edu

Will Glover

University of Nebraska – Lincoln
Email: wglover2@huskers.unl.edu

Rishi Krishna

University of Nebraska – Lincoln
Email: rkrishna2@huskers.unl.edu

Michael Dejournett

University of Nebraska – Lincoln
Email: mdejournett2@huskers.unl.edu

Amy Nguyen

University of Nebraska – Lincoln
Email: anguyen105@huskers.unl.edu

Maya Wilson

University of Nebraska – Lincoln
Email: mwilson75@huskers.unl.edu

Krishnaraj Ganesan

University of Nebraska – Lincoln
Email: krishnarajganesan@nebraska.edu

Abstract—Curia is a campus event search system for University of Nebraska–Lincoln that parses natural-language queries into keywords and structured temporal bounds. This paper studies how far the underlying language model can be reduced while preserving keyword expansion and temporal grounding under tight CPU latency and memory budgets.

We evaluate seven model candidates—a remote 27B reference (Gemini), a local 8B baseline (Ollama llama3), and five Hugging Face models (248M–1.5B)—on three benchmarks: a 72-query temporal correctness suite, a 72-query robustness correctness suite using perturbed queries, and a latency-stability suite that repeats each robustness query three times to measure cross-run variance. All runs use a CUDA-backed local evaluation harness to isolate model capacity from hardware variation; CPU-constrained deployment is the target environment. On the temporal suite, Gemini achieves perfect extraction (100% parse, 100% date and time exact-match). Among local models, Ollama llama3 (8B) parses all queries but attains only 20.83% date exact-match at roughly 3.6s mean latency; Qwen2.5-1.5B matches Gemini on date exact-match and reaches 95.83% time exact-match at lower latency. The two T5-family encoder-decoder models return structurally valid outputs but no correct temporal fields, indicating a task-specific capacity threshold near 1B parameters. On the robustness suite, three of seven models pass the correctness gate—Gemini, Ollama 8B, and Qwen2.5-1.5B—while Qwen2.5-0.5B and TinyLlama-1.1B fail on keyword F1, demonstrating that keyword extraction quality must be assessed independently across evaluation conditions. The latency-stability suite finds all seven models within variance and tail-latency bounds, with Qwen2.5-1.5B exhibiting zero cross-run variance in both parse and keyword outputs.

Index Terms—Lightweight Models, Temporal Grounding, Information Retrieval, Query Parsing, Edge Inference

I. INTRODUCTION

Curia’s natural language search system depends on two query-understanding capabilities: extracting and expanding intent-bearing keywords for retrieval, and grounding temporal expressions into structured date and time bounds. A user typing “free food next Thursday after 3pm” has simultaneously specified a topic, a relative date anchor, and a time lower

bound. Converting this free-form input into structured filters drives retrieval quality.

In practice, the search must operate under tight memory and latency budgets, which pushes deployment toward smaller models. This raises a central design tension: how far can the underlying model be reduced before these extraction behaviors degrade enough to harm retrieval? Prior work on neural scaling laws [1], [2] suggests that capabilities can decline unevenly as capacity decreases, and the effect is often amplified when the task requires reliable structured output.

This paper studies that tension directly in Curia, a campus event search system for UNL. Rather than targeting architectural novelty, we measure how model size affects the two core tasks that drive retrieval: keyword expansion and temporal grounding. We focus on four research questions:

- 1) Does the gap between the two tasks, temporal grounding and keyword expansion, begin to diverge as model size decreases?
- 2) At what point can our structured prompt no longer produce usable results?
- 3) How stable are these results, and does the stability substantively change as model size decreases?
- 4) What model size offers the best tradeoff among stability, accuracy, and latency?

Our experiments show asymmetric degradation across tasks and datasets. Temporal grounding is more fragile than keyword expansion under perturbation, and performance does not scale monotonically with parameter count alone. In particular, Qwen2.5-1.5B is the only local model that passes the correctness gate on both evaluation suites, while smaller models either fail temporal grounding or fall below the keyword F1 threshold under perturbed queries.

This paper makes three contributions:

- **Task-specific size analysis:** An empirical comparison of seven model candidates (248M to 27B) on keyword expansion and temporal grounding within the same event-

search pipeline, evaluated on two correctness benchmarks of 72 labeled queries each.

- **Failure characterization:** Documentation of how and where temporal extraction breaks down as model capacity decreases, showing that structured-output correctness is unrecoverable below 1B parameters through prompt engineering or fallback heuristics alone.
- **Deployment guidance:** Evidence that keyword expansion is more size-resilient than temporal grounding, with practical implications for model selection at different deployment scales.

II. RELATED WORK

Our work sits at the intersection of neural scaling, temporal information extraction, in-context learning, and small-model deployment. We review the most relevant prior work in each area below.

A. Neural Scaling Laws

Kaplan et al. [1] showed that language model loss scales as a power law with model size, dataset size, and compute across seven orders of magnitude. Hoffmann et al. [2] refined this, finding that many widely-used models are undertrained relative to their parameter count and that tokens and parameters should be scaled together. Both results bear directly on our study: if performance degrades predictably with size, we should be able to identify a parameter range where structured extraction breaks down—and our experiments show that range is sharper than a smooth power law would suggest.

B. Temporal Extraction and Hybrid Retrieval

HeidelTime [3] is a rule-based system for extracting and normalizing temporal expressions using hand-crafted grammars. It works well on standard patterns but needs domain-specific tuning for new text types. We use `dateparser` in a similar capacity as a fallback: when the LLM returns malformed JSON, it tries to recover any parseable date strings. The catch is that it only fires on parse failures—when the model returns valid JSON with wrong dates, which is the main failure we document, the fallback never gets a chance to run. For keyword expansion, Reimers and Gurevych [4] showed that Siamese BERT networks produce dense sentence embeddings well-suited for semantic similarity search. We use `all-MiniLM-L6-v2`, a compact Sentence-BERT model, to retrieve semantically related domain terms and append them to the LLM’s keyword output at under 50 ms per query.

C. Few-Shot and In-Context Learning

Brown et al. [5] showed that large language models can follow new task specifications from a handful of in-context examples alone, no weight updates needed, and that this ability improves with scale. We rely on this throughout: all models receive the same in-context exemplars (4 temporal, 3 keyword) under an identical prompt contract. Our results quantify the scale dependency: the 27B Gemini reference reliably produces correct temporal bounds across the suite, while local models

show heterogeneous behavior. The Qwen2.5-1.5B run achieves parity with Gemini on strict date exact-match, whereas the Ollama 8B baseline attains modest strict-date performance (20.83%) and lower time accuracy (37.50%). Several smaller candidates fail to produce consistent temporal bounds, suggesting a capacity threshold for dependable structured temporal grounding under CPU-constrained execution.

D. Small and Specialized Language Models

Several groups have looked at whether small models are viable substitutes for large ones in narrow domains. Grangier et al. [6] propose resampling pretraining data toward a target domain using importance sampling, alongside a projected-network architecture that compresses a large model into a smaller one at inference time. Belcak et al. [7] make the broader argument that SLMs are a natural fit for the repetitive, single-purpose subtasks that show up in production agentic pipelines. Fan et al. [8] report that sub-3B models can compete on time-series forecasting when given descriptive statistical prompts and an adaptive fusion layer. These papers make a reasonable case that small models are worth taking seriously for specialized tasks. We test that idea on structured temporal grounding and find the gap to the 27B reference model is not closeable in the sub-2B range under our evaluation conditions.

III. METHODOLOGY

A. Evaluation Design

The benchmark comprises three evaluation runs over a fixed seven-model matrix and two dataset presets. The primary correctness runs cover two 72-query datasets: `rigorous-temporal`, a benchmark focused on temporal grounding with labeled date/time bounds, and `rigorous-robustness`, a dataset of 72 perturbed queries testing stability under adversarial and edge-case phrasing. A third *latency-stability* run repeats each query three times (216 total query executions per model) to measure cross-run variance in parse behavior, keyword output, and tail latency; both `rigorous-temporal` and `rigorous-robustness` are covered. The `all-events` preset covering the full event catalog exceeds 10 hours on consumer machines and is excluded to prioritize reproducibility.

The protocol keeps the inference backend and execution mode explicit. All runs use the local Hugging Face backend on CUDA GPU to ensure consistent, hardware-isolated latency comparisons; CPU-constrained deployment is the intended production environment. The benchmark measures the interaction between model capacity and the Curia query-understanding pipeline under a stable test harness, permitting clean attribution of performance differences to model scale rather than implementation variation.

B. Task Outputs

Given a natural language query q , Curia produces two outputs:

- **Keywords:** An expanded set of intent-bearing terms for retrieval matching.
- **Temporal bounds:** A JSON object with four fields — `date_from`, `date_to`, `time_from`, `time_to` — or null when no constraint is present.

For example, “morning concerts next weekend” should yield:

```
{
  "date_from": "2026-05-02",
  "date_to": "2026-05-03",
  "time_from": "06:00",
  "time_to": "12:00",
  "keywords": ["concert", "music",
               "performance", "band", "show"]
}
```

C. Scoring Framework

The correctness suite summarizes each model with task-level extraction metrics. For a query q , the benchmark records whether the model returns parseable JSON, whether the date and time fields match the labeled targets, and whether the predicted keywords align with the expected keyword set. The reported metrics are: parse success rate, date exact-match rate, time exact-match rate (temporal suite) or time partial-match rate (robustness suite), and keyword F1.

Keyword F1 and parse success are always computed over all 72 queries. Date exact-match is computed over the labeled subset of each dataset: the temporal dataset carries 72 date-labeled and 48 time-labeled queries; the robustness dataset carries 48 date-labeled and 24 time-labeled queries. Percentage rates reported in tables therefore have different effective denominators depending on the column. On the robustness suite, no model achieves nonzero time exact-match (the robustness queries target partial rather than strict time alignment), so we report time partial-match in Table III instead.

A binary *correctness gate* determines overall model usability with thresholds: parse ≥ 0.70 , keyword F1 ≥ 0.05 , date exact ≥ 0.50 , and time partial ≥ 0.50 . A model passes only if all four criteria are met. The goal of these gates is to identify models that remain deployable under minimum service-level requirements; results near the threshold boundaries are noted where relevant.

A secondary *latency-stability gate* evaluates operational reliability across repeated executions, with six thresholds: p95 latency $\leq 15,000$ ms, timeout rate ≤ 0.10 , error rate ≤ 0.25 , parse success standard deviation ≤ 0.10 , keyword F1 standard deviation ≤ 0.10 , and p95 latency standard deviation $\leq 4,000$ ms. A model passes this gate only when all six criteria are satisfied across the three repeated runs per query.

The central analytical question is whether keyword expansion and temporal grounding degrade together as model size decreases, or diverge. A model that retains strong keyword F1 but fails date extraction, or vice versa, indicates that the two capabilities have different capacity thresholds—a finding that has direct implications for feature selection at deployment.

D. Inference Paths

a) *Gemini Remote Path (27B baseline)*: The reference path sends a single structured prompt to `gemma-3-27b-it` via the `google-genai` SDK. The prompt includes the current date and requests a JSON object containing temporal fields, keyword expansion, and retrieval-oriented output. This path serves as the upper-bound reference for quality-versus-latency comparisons and gate-pass analysis reported in the evaluation.

b) *Local Model Path*: The local HuggingFace candidates execute with automatic device selection: CUDA when available, falling back to CPU otherwise. The Ollama baseline (`llama3:latest`) runs through Ollama’s local inference server. Each model is evaluated with the same structured prompt contract, the same top-level output schema, and the same decoding settings so that differences in output quality are attributable to model capacity rather than prompt redesign. The benchmark runner preserves a fixed prompt interface across all local candidates; the only variation is the model identity and the execution backend.

c) *Fallback Layer*: For malformed JSON outputs, the benchmark applies a fallback policy: (1) attempt `dateparser` to recover parseable date strings when JSON parsing fails; (2) time-of-day keyword lookup for time fields (morning $\rightarrow$ 06:00–12:00, afternoon $\rightarrow$ 12:00–17:00, evening $\rightarrow$ 17:00–21:00); (3) null bounds if both fail. This recovery layer applies uniformly across all models, allowing divergence in performance to be attributed to model capacity. However, the fallback only recovers from malformed JSON; when a model returns valid JSON with incorrect temporal values (e.g., wrong date arithmetic), the fallback does not intervene, and the error is recorded as-is in the results.

E. Event Scoring

The backend ranks events by weighted field score:

$$\text{score}(e, Q) = \sum_{t \in Q} w_f \cdot \mathbf{1}[t \in \text{field}_f(e)] \cdot \text{pos}(t, Q)$$

where $w_f \in \{4, 2, 1\}$ for name, venue/category, and description/tags respectively, and $\text{pos}(t, Q)$ is a position bonus giving higher weight to terms that appear earlier in the query. Temporal bounds are applied as hard pre-filters before scoring.

F. Models Evaluated

Table I lists the seven model candidates evaluated in the benchmark harness.

G. Model Selection Rationale

The benchmark model set was selected to probe the parameter-performance frontier at key intervals and to include both instruction-tuned decoder-only and encoder-decoder architectures. `gemma-3-27b-it` serves as the remote reference model—a production-grade instruction-tuned 27B decoder-only model via the Gemini API that reliably solves both tasks. `llama3:latest` (8B) is included as a local baseline available via Ollama, bridging between the Gemini

TABLE I
MODEL CANDIDATES

Model	Backend	Params
gemma-3-27b-it	Gemini API	27B
llama3:latest	Ollama local	8B
Qwen2.5-0.5B	HF local	500M
Qwen2.5-1.5B	HF local	1.5B
TinyLlama-1.1B-Chat	HF local	1.1B
google/flan-t5-base	HF local	250M
LaMini-Flan-T5-248M	HF local	248M

scale and smaller local candidates while remaining practical for CPU inference.

The local decoder-only candidates span the sub-1B region: Qwen2.5-0.5B (500M, instruction-tuned, chat-templated) is the active default in production due to its speed and performance on lower-range consumer machines; Qwen2.5-1.5B (1.5B, same architecture) was included to test whether crossing into the 1B range improves temporal reasoning at the cost of latency; and TinyLlama-1.1B-Chat (1.1B, a multi-turn chat-optimized model) was included to evaluate whether alternative chat architectures in the 1B range escape the structured-output ceiling. The encoder-decoder candidates are smaller T5 variants: flan-t5-base (250M, instruction-tuned via FLAN) and LaMini-Flan-T5-248M (248M, a distilled T5 trained for abstractive tasks). These were included to determine whether the seq2seq inductive bias offers advantages over decoder-only scaling for this task.

IV. RESULTS

A. Benchmark Design

We evaluate the same seven-model matrix on two 72-case correctness datasets (rigorous-temporal and rigorous-robustness) and two latency-stability datasets, one per correctness suite (three repeated runs each, 216 total executions per model). The temporal set emphasizes explicit temporal grounding; the robustness set introduces perturbed and edge-case query forms. All runs use the same prompt contract, decoding settings, and CUDA local backend configuration where applicable.

All numeric values in this section are taken directly from the benchmark run artifacts: `report_data/correctness/temporal/summary.json`, `report_data/correctness/robustness/summary.json`, `report_data/latency/temporal/summary.json`, and `report_data/latency/robustness/summary.json` together with the corresponding `gate_results.json` files. End-to-end retrieval metrics (top- N event hit rate, MRR) are recorded in the run artifacts but are all zero across both datasets and all models, because the events corpus was not available in the evaluation environment; only extraction metrics are reported here.

B. Accuracy on the Temporal Dataset

Table II reports the canonical extraction metrics from the temporal benchmark run. Parse success is the fraction of queries returning valid JSON; keyword F1 is the overlap between predicted and expected keyword sets; both are aggregated over all 72 queries. Date exact-match is computed over all 72 date-labeled queries. Time exact-match is computed over the 48 queries that carry time labels; percentages in that column therefore have an effective denominator of 48.

The remote Gemini reference achieves perfect extraction rates on this benchmark (100% parse, date, and time exact-match) under the structured prompt contract. Among local candidates, the picture is heterogeneous: Ollama 8B parses every query but attains only 20.83% strict date exact-match and 37.50% time exact-match, while achieving a comparatively high keyword F1 (0.219). Qwen2.5-1.5B attains full date exact-match parity with Gemini and near-parity on time exact-match (95.83%), at a lower mean latency than Gemini. The two T5 variants at approximately 250M parameters produce no correct temporal fields on this suite despite non-trivial parse success, indicating a capacity floor for this task below 1B parameters.

C. Accuracy on the Robustness Dataset

Table III summarizes the same metrics on rigorous-robustness. This dataset has 48 date-labeled and 24 time-labeled cases out of 72 total queries; rate denominators differ by column accordingly. Time partial-match is reported rather than time exact-match because no model achieves nonzero time exact-match on this suite—the robustness queries are designed around partial time-range agreement rather than strict field equality.

Relative to temporal, the robustness suite reveals a differentiated gate profile. Gemini and Qwen2.5-1.5B maintain full date exact-match across both suites. Ollama 8B, which failed the temporal gate on date exact-match (20.83%), passes the robustness gate at exactly the 50% threshold on date exact-match, paired with a high keyword F1 (0.278). TinyLlama and Qwen2.5-0.5B each achieve 100% date exact-match on robustness but fall below the keyword F1 threshold (0.0478 and 0.0415, respectively, against the 0.05 gate minimum), resulting in gate failures despite strong temporal extraction.

D. Latency-Stability Results

Table IV reports the latency-stability suite across both rigorous-temporal and rigorous-robustness, each conducted with three runs per query (216 total executions per model per suite). The reported p95, standard deviations, and error rates are aggregated across all three runs. All seven models pass the latency-stability gate on both suites.

Across both suites the variance profiles are consistently low. Qwen2.5-1.5B exhibits the tightest p95 standard deviation among local HuggingFace models on both temporal (25.8ms) and robustness (110.8ms), with zero parse and keyword F1 variance in each case. Gemini is similarly stable, with p95 stddev of 47.4ms and 36.8ms on temporal and robustness

TABLE II
TEMPORAL CORRECTNESS SUITE: EXTRACTION ACCURACY (72 QUERIES). DATE EXACT OVER ALL 72 LABELED CASES; TIME EXACT OVER 48 TIME-LABELED CASES; KW F1 AND PARSE OVER ALL 72.

Model	Params	Parse	Date Exact	Time Exact	KW F1	Mean Latency (ms)
gemma-3-27b-it (Gemini)	27B	100%	100%	100%	0.0748	4,523
llama3:latest (Ollama)	8B	100%	20.83%	37.50%	0.219	3,569
Qwen2.5-1.5B	1.5B	100%	100%	95.83%	0.0602	2,935
TinyLlama-1.1B-Chat	1.1B	98.61%	98.61%	52.08%	0.170	2,159
Qwen2.5-0.5B	500M	95.83%	81.94%	58.33%	0.0874	3,519
google/flan-t5-base	250M	86.11%	0%	0%	0.000	817
LaMini-Flan-T5-248M	248M	98.61%	0%	0%	0.000	845

TABLE III
ROBUSTNESS CORRECTNESS SUITE: EXTRACTION ACCURACY (72 QUERIES). DATE EXACT OVER 48 DATE-LABELED CASES; TIME PARTIAL OVER 24 TIME-LABELED CASES; KW F1 AND PARSE OVER ALL 72.

Model	Params	Parse	Date Exact	Time Partial	KW F1	Mean Latency (ms)
gemma-3-27b-it (Gemini)	27B	100%	100%	79.17%	0.0769	4,581
llama3:latest (Ollama)	8B	100%	50.00%	62.50%	0.278	3,574
Qwen2.5-1.5B	1.5B	100%	100%	87.50%	0.0994	2,834
TinyLlama-1.1B-Chat	1.1B	100%	100%	95.83%	0.0478	1,339
Qwen2.5-0.5B	500M	100%	100%	62.50%	0.0415	1,791
google/flan-t5-base	250M	93.06%	0%	0%	0.000	842
LaMini-Flan-T5-248M	248M	98.61%	0%	0%	0.000	1,001

TABLE IV
LATENCY-STABILITY SUITE ($72 \times 3 = 216$ EXECUTIONS PER MODEL PER DATASET). GATE THRESHOLDS: $p95 \leq 15,000$ MS; TIMEOUT RATE ≤ 0.10 ; ERROR RATE ≤ 0.25 ; PARSE STDDEV ≤ 0.10 ; KW F1 STDDEV ≤ 0.10 ; $p95$ STDDEV $\leq 4,000$ MS. ALL MODELS PASS BOTH SUITES.

Model	Params	p95 (ms)	p95 Std (ms)	Parse Std	KW F1 Std	Error Rate	Gate
<i>(a) Temporal Dataset</i>							
gemma-3-27b-it (Gemini)	27B	4,993	47.4	0.000	0.001	0.000	Pass
llama3:latest (Ollama)	8B	3,877	54.4	0.000	0.004	0.000	Pass
Qwen2.5-1.5B	1.5B	3,465	25.8	0.000	0.000	0.000	Pass
TinyLlama-1.1B-Chat	1.1B	2,187	26.3	0.000	0.000	0.014	Pass
Qwen2.5-0.5B	500M	2,859	33.0	0.000	0.000	0.014	Pass
google/flan-t5-base	250M	1,085	28.7	0.000	0.000	0.139	Pass
LaMini-Flan-T5-248M	248M	1,015	19.7	0.000	0.000	0.014	Pass
<i>(b) Robustness Dataset</i>							
gemma-3-27b-it (Gemini)	27B	4,695	36.8	0.000	0.001	0.000	Pass
llama3:latest (Ollama)	8B	3,901	50.3	0.007	0.004	0.005	Pass
Qwen2.5-1.5B	1.5B	3,372	110.8	0.000	0.000	0.000	Pass
TinyLlama-1.1B-Chat	1.1B	1,888	91.0	0.000	0.004	0.000	Pass
Qwen2.5-0.5B	500M	2,656	40.4	0.000	0.000	0.000	Pass
google/flan-t5-base	250M	985	14.0	0.000	0.000	0.069	Pass
LaMini-Flan-T5-248M	248M	1,616	58.2	0.000	0.000	0.014	Pass

respectively, and zero parse variance—consistent with a deterministic remote API at temperature zero. Ollama records one timeout across 216 robustness executions (error rate 0.5%) but runs cleanly on temporal (error rate 0.0%), with minor keyword F1 variance (stddev 0.004) on both suites.

The most notable cross-suite difference is in error rate for the encoder-decoder models. On robustness, flan-t5-base records a 6.9% error rate; on temporal this rises to 13.9%, suggesting that the more complex temporal phrasing of rigorous-temporal places greater strain on the seq2seq architecture despite its low per-token compute cost. TinyLlama-1.1B and Qwen2.5-0.5B each introduce a 1.4% error rate on temporal (zero on robustness for both), consistent

with the occasional extreme-latency outliers observed in the single-run correctness data; across three runs the p95 values remain stable (2,187 ms and 2,859 ms respectively), confirming these as rare outlier events rather than systematic degradation.

E. Research Questions (RQ1–RQ4)

1) RQ1: Do temporal grounding and keyword expansion diverge as size decreases?: **Determination: Yes, and the divergence is visible in both datasets.**

On rigorous-temporal, Ollama 8B attains the highest keyword F1 (0.219) of any model, but the lowest date exact-match among parseable models (20.83%), while Qwen2.5-1.5B achieves near-perfect temporal grounding (100% date

exact, 95.83% time exact) at a keyword F1 of only 0.0602. On `rigorous-robustness`, the same inversion holds: Ollama leads in keyword F1 (0.278) while posting 50.00% date exact-match, whereas Qwen2.5-1.5B again achieves 100% date exact at a keyword F1 of 0.0994. This indicates the two tasks do not degrade together under downscaling, and that a model strong at keyword expansion is not necessarily strong at temporal grounding, or vice versa.

One counterintuitive result in Table II is that Gemini—the 27B reference—attains a lower keyword F1 (0.0748) than Ollama 8B (0.219) and TinyLlama 1.1B (0.170). Gemini’s keyword precision against the labeled set is 0.041 with recall of 0.426, while Ollama achieves precision 0.136 and recall 0.685. The gap reflects that Gemini generates a broader, less label-aligned synonym set: although the generated terms may be semantically valid, they do not overlap with the curated label vocabulary, suppressing F1 despite producing non-trivial recall. This is a property of the label set construction rather than direct evidence of inferior keyword generation by the larger model.

2) *RQ2: At what point does the structured prompt stop producing usable results?:* **Determination: Below 1B parameters for temporal grounding on both datasets; for keyword expansion, the threshold varies by suite and falls near 0.05 F1 for several sub-1.5B models.**

Applying the correctness gates from `gate_results.json`: the temporal suite passes 4/7 models (Gemini, Qwen2.5-1.5B, TinyLlama-1.1B, and Qwen2.5-0.5B), while the robustness suite passes 3/7 (Gemini, Ollama 8B, and Qwen2.5-1.5B). The only models to pass both correctness gates are Gemini and Qwen2.5-1.5B.

Both T5-family models fail both datasets with 0% date exact-match, 0% time, and zero keyword F1, confirming non-usable extraction despite mostly parseable output. Three gate boundary cases are notable. Ollama 8B passes the robustness gate at exactly the 50% date exact-match threshold—the narrowest possible passing margin—indicating that its robustness performance is marginal. TinyLlama-1.1B passes the temporal gate but fails the robustness gate not on temporal metrics (100% date exact on both suites) but on keyword F1 (0.0478, a deficit of 0.002 below the 0.05 threshold); this is the closest gate failure in the set and illustrates how a single weak dimension can disqualify an otherwise strong model. Qwen2.5-0.5B passes the temporal correctness gate but fails the robustness gate on keyword F1 (0.0415, a deficit of 0.009), despite achieving 100% date exact-match on the robustness set.

3) *RQ3: How stable are results as size decreases?:* **Determination: All models pass the latency-stability gate on both suites; stability metrics do not correlate monotonically with parameter count, and cross-suite variance profiles are broadly consistent.**

The latency-stability suite provides direct cross-run variance measurements for both `rigorous-temporal` and `rigorous-robustness`. All seven models satisfy every threshold of the latency-stability gate on both suites, indicating that none exhibit catastrophic instability under three

repeated runs. Qwen2.5-1.5B shows the most consistent behavior among local models: zero variance in parse success and keyword F1 on both suites, with p95 standard deviations of 25.8 ms (temporal) and 110.8 ms (robustness). Gemini achieves near-zero variance across all stability metrics on both suites (p95 stddev 47.4 ms temporal; 36.8 ms robustness), consistent with a deterministic remote API at temperature zero.

The principal cross-suite differences involve error rates rather than latency variance. Ollama records one timeout on the robustness suite (error rate 0.5%) but runs without error on temporal, suggesting that perturbed query forms are marginally harder for its inference path. TinyLlama-1.1B and Qwen2.5-0.5B each produce a 1.4% error rate on temporal (zero on robustness), corresponding to rare extreme-latency outlier queries; the p95 values across three runs (2,187 ms and 2,859 ms respectively) confirm these are isolated events rather than systematic degradation. The most pronounced cross-suite shift is in `flan-t5-base`’s error rate, which doubles from 6.9% on robustness to 13.9% on temporal—consistent with the `seq2seq` architecture being sensitive to the more complex temporal phrasing in `rigorous-temporal`—yet remains within the 25% gate threshold. Stability, like correctness, therefore varies by model-dataset interaction rather than by parameter count alone.

4) *RQ4: What model size offers the best tradeoff among stability, accuracy, and latency?:* **Determination: Qwen2.5-1.5B is the strongest local operating point across all three evaluation dimensions; no sub-1.5B local model satisfies all three suites simultaneously.**

Qwen2.5-1.5B is the only local model to pass both correctness gates and the latency-stability gate. On temporal correctness it achieves 100% date exact-match, 95.83% time exact-match, and a mean latency of 2,935 ms. On robustness correctness it maintains 100% date exact-match and 87.50% time partial-match at 2,834 ms mean latency. On the latency-stability suite it exhibits zero cross-run variance in parse and keyword outputs, with a p95 standard deviation of 110.8 ms.

Gemini passes all three suites but operates at higher latency (4,523 ms and 4,581 ms mean on temporal and robustness respectively), incurs API costs, and requires network availability—properties that conflict with the deployment constraints motivating this study. Among models that pass only one correctness gate, TinyLlama-1.1B is the most notable near-miss: it passes the temporal gate and achieves full date exact-match on robustness, but its keyword F1 on robustness (0.0478) falls marginally below threshold. If keyword expansion quality can be improved through prompt tuning, TinyLlama’s low latency (1,338 ms mean on robustness, lowest among decoder-only models) makes it an attractive candidate for further investigation. Qwen2.5-0.5B fails the robustness correctness gate and therefore cannot be recommended as a deployment alternative at this evaluation.

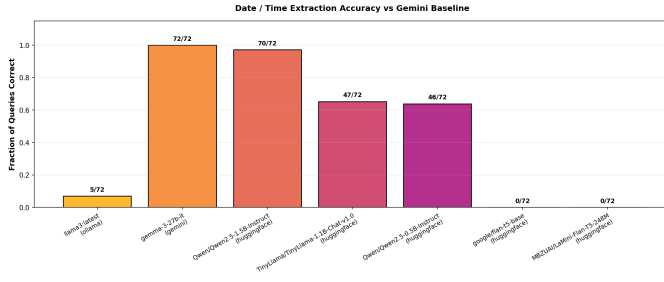


Fig. 1. Temporal-suite retrieval overlap with the Gemini baseline. Each bar shows the fraction of queries on which a model’s top-10 event results match Gemini’s, reflecting the compounded effect of date correctness, time correctness, and keyword ranking. Models with higher date exact-match in Table II correspondingly show higher overlap here; T5-family models produce zero overlap due to complete temporal extraction failure.

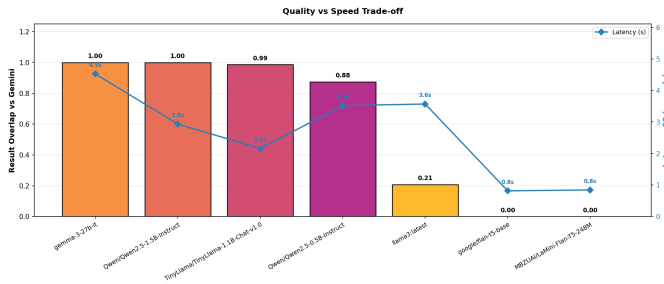


Fig. 2. Temporal-suite quality-vs-speed trade-off. Bars show retrieval overlap with the Gemini baseline (left axis); the line shows mean latency in seconds (right axis). Models are sorted by quality. Qwen2.5-1.5B and Gemini achieve the highest overlap; T5-family models are the fastest but produce no useful temporal output. Note: the quality axis reflects Jaccard overlap with Gemini’s retrieved event set, not direct extraction accuracy.

F. Supporting Visualizations

V. CONCLUSION

We report a systematic efficiency study on model-size minimization for event query grounding in the Curia campus search system. Across temporal and robustness correctness suites and a latency-stability suite, results confirm asymmetric degradation between keyword expansion and temporal grounding, with gate outcomes that depend on both model capacity and evaluation conditions.

Key findings:

- **Task divergence is consistent:** Higher keyword F1 does not imply stronger temporal grounding; the two capabilities must be evaluated separately, as demonstrated by the Ollama–Qwen inversion on both suites.
- **Usability threshold is task- and suite-dependent:** The 250M-class T5-family runs are non-usable on both correctness suites. Robustness narrows the set of passing models from 4/7 to 3/7; only Gemini and Qwen2.5-1.5B pass both.
- **Stability does not track parameter count:** All models pass the latency-stability gate on the robustness suite.

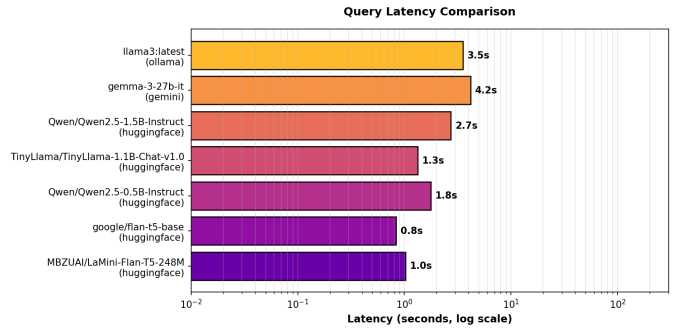


Fig. 3. Latency-stability suite: latency distributions across three repeated runs on the robustness dataset (216 executions per model). The temporal suite exhibits an analogous distribution. The narrow p95 spread for most models reflects the low latency variance reported in Table IV; T5-family models occupy the sub-1,100 ms range while decoder-only models cluster between 1,300–5,000 ms.

Qwen2.5-1.5B is the most consistent local model (zero cross-run variance in parse and keyword outputs), while the T5-family models exhibit flat latency despite failing all correctness criteria.

- **Best local operating point:** Qwen2.5-1.5B is the unique local model satisfying all three evaluation suites, combining full date exact-match on both correctness datasets with zero correctness variance in the latency-stability run.
- **Fallbacks remain limited:** Rule-based fallbacks recover explicit date strings from malformed JSON but do not correct semantically incorrect temporal reasoning in otherwise parseable outputs.

A. Future Work

- **Quantized local models:** GGUF int4/int8 variants of Qwen2.5-1.5B and Qwen2.5-3B to improve the local accuracy-latency frontier without full-precision CPU inference.
- **Confidence-gated fallback:** Use token log-probabilities to decide when to trust LLM output versus fall back to rules, reducing unnecessary fallback invocations.
- **Recurring event queries:** Extend temporal extraction to handle recurring expressions (“every Monday”, “weekly seminars”).
- **Expanded benchmark:** Add typo-robustness and cross-domain query variants; add a retrieval evaluation dataset with labeled relevant events to enable end-to-end hit-rate measurement currently unavailable due to corpus constraints.
- **TinyLlama keyword improvement:** Investigate whether prompt tuning or keyword post-processing can raise TinyLlama-1.1B’s keyword F1 above the 0.05 gate threshold, given its otherwise strong temporal accuracy and lowest decoder-only latency.

REFERENCES

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural

- language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022.
 - [3] J. Strötgen and M. Gertz, “HeidelTime: High quality rule-based extraction and normalization of temporal expressions,” in *Proceedings of the 5th International Workshop on Semantic Evaluation*, 2010, pp. 321–324.
 - [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.
 - [5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
 - [6] D. Grangier, A. Katharopoulos, P. Abhin, and A. Hannun, “Need a small specialized language model? Plan Early!” *arXiv preprint arXiv:2402.01093*, 2024.
 - [7] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, “Small language models are the future of agentic AI,” *arXiv preprint arXiv:2506.02153*, 2025.
 - [8] H. Fan, B. Li, Y. Weng, and S. Zhou, “Small but mighty: Enhancing time series forecasting with lightweight LLMs,” *arXiv preprint arXiv:2503.03594*, 2025.